

Mallocator 설계 문서

이름: 백서연

학번: 20221197

이메일: yorange50@gmail.com

1. 개요

이 프로젝트는 C에서 동적 메모리 할당을 위한 malloc, free, realloc 함수를 직접 구현하는 과제입니다. 명시적 가용 리스트(explicit free list)를 사용하여 메모리 효율성과 할당 속도를 모두 개선하고자 하였으며, first-fit 전략 및 즉시 병합(immediate coalescing) 기법을 적용했습니다.

2. 전반적 설계 구조

사용한 전략

- **명시적 가용 리스트**: free block만을 따로 리스트로 연결하여, malloc 시 탐색 속도 개선
- **First-fit**: 가용 리스트에서 처음 맞는 블록을 사용
- **Immediate coalescing**: free 시점에 이전/다음 블록과 즉시 병합
- **8바이트 정렬**: 모든 payload는 8바이트 정렬
- **헤더/풋터를 이용한 메모리 블록 구조**

Block 구조

| HEADER | PREV_FREE | NEXT_FREE | ... Payload ... | FOOTER |

(Free 블록의 경우: PREV_FREE / NEXT_FREE 포함)

(Allocated 블록의 경우: Payload만 있음)

3. 전역 변수 설명

```
static char *heap_listp = NULL;
```

```
static char *free_listp = NULL;
```

- heap_listp: 힙의 시작 주소를 가리킵니다. mem_sbrk() 호출 이후 heap 영역 조작 시 기준 포인터입니다.
- free_listp: 가용 블록 리스트의 시작 주소입니다. 명시적 리스트 탐색 시 사용됩니다.

4. 매크로 정의 요약

매크로 이름	설명
WSIZE, DSIZE	워드/더블워드 크기 (4/8바이트)
PACK(size, alloc)	블록의 크기와 할당 여부를 비트로 패킹
GET(p), PUT(p, val)	포인터를 통한 읽기/쓰기
GET_SIZE(p), GET_ALLOC(p)	헤더/풋터에서 블록 크기와 상태 추출
HDRP(bp), FTRP(bp)	헤더/풋터 주소 계산
NEXT_BLKP(bp), PREV_BLKP(bp)	블록 간 이동
NEXT_FREE(bp), PREV_FREE(bp)	명시적 가용 리스트의 연결 포인터

5. 주요 함수 설명

int mm_init(void)

- 초기 heap 생성 및 prologue/epilogue 설정
- 초기 heap 확장을 extend_heap()으로 수행

static void *extend_heap(size_t words)

- 힙 영역을 확장하여 새 블록을 생성
- 새로 추가된 블록을 coalesce()로 병합 시도

static void *coalesce(void *bp)

- 인접 블록과의 병합 수행 (4가지 case)
- 병합된 블록을 가용 리스트에 삽입

void *mm_malloc(size_t size)

- size를 정렬 및 최소 블록 크기 적용
- 가용 리스트에서 find_fit으로 적합한 블록 찾고 place() 호출
- 없으면 extend_heap() 호출

void mm_free(void *ptr)

- 주어진 블록을 free로 표시하고 coalesce() 호출

void *mm_realloc(void *ptr, size_t size)

- 메모리 확장이 필요한 경우:
 - 다음 블록이 free이고 병합으로 충분할 경우 in-place 확장
 - 그렇지 않으면 새 공간 할당 후 복사 및 기존 블록 free

static void place(void *bp, size_t asize)

- 블록을 할당하고, 필요시 분할하여 남은 부분은 free로 설정

static void *find_fit(size_t asize)

- 가용 리스트에서 처음 맞는 free 블록을 반환 (first-fit)

static void insert_to_list(void *bp)

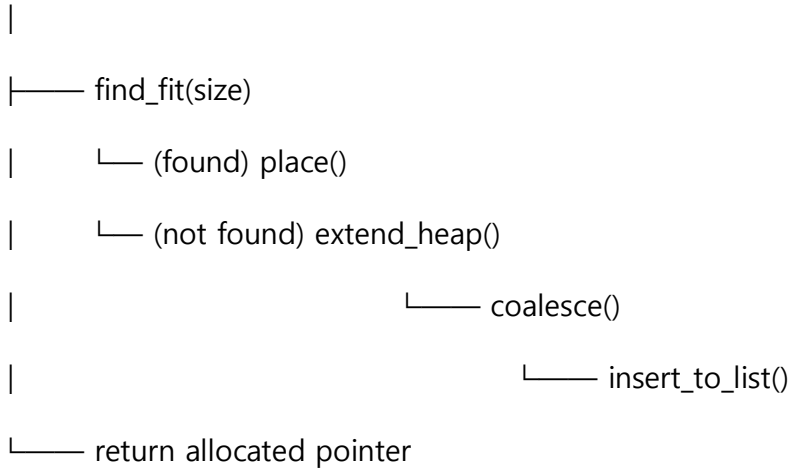
- 새로 free된 블록을 가용 리스트 맨 앞에 삽입

static void remove_from_list(void *bp)

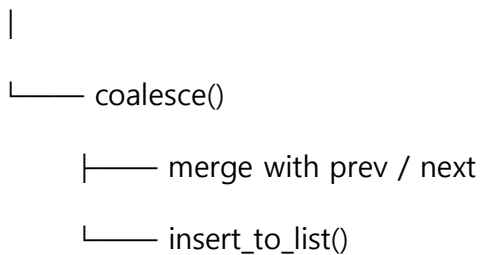
- 가용 리스트에서 해당 블록 제거

6. 설계도 (구조 흐름도)

mm_malloc(size)



mm_free(ptr)



7. 최적화 및 성능 고려 사항

- realloc의 in-place 확장 구현: 성능 개선
- 작은 블록은 분할해서 사용
- coalesce 시 가용 리스트 정리
- 명시적 리스트를 이용하여 탐색 효율 증가

8. 기타 참고 사항

- 모든 블록은 8바이트 정렬 보장
- mm.c는 cspro 서버에서 컴파일 및 테스트 완료

- 가용 리스트에 더블 링크드 리스트 사용 (양방향 연결)
 - 동작 확인을 위해 -V, -v, -f 옵션으로 mdriver 테스트 수행
-